

LAPORAN PRAKTIKUM STRUKTUR DATA

Stack



Oleh :

MUHAMMAD ARIF

NIM 2511532017

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 18 April 2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan praktikum “**Stack**” ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai dokumentasi dan pemahaman hasil kegiatan praktikum yang bertujuan untuk memahami bagaimana konsep dari Stack. Penulis juga terbuka untuk menerima saran dan kritik guna perbaikan di masa mendatang.

Penulis mengucapkan terima kasih sebesar-besarnya kepada:

1. DR. Wahyudi, S.T, M.T selaku pembimbing mata kuliah basis data, yang telah memberikan bimbingan, ilmu, dan motivasi selama proses pembelajaran.

Padang, 18 April 2026

Muhammad Arif

DAFTAR PUSTAKA

LAPORAN PRAKTIKUM STRUKTUR DATA.....	i
KATA PENGANTAR.....	ii
DAFTAR PUSTAKA.....	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat	2
BAB II PEMBAHASAN	3
2.1 Landasan teori	3
2.2 Langkah Pengerjaan	3
BAB III KESIMPULAN.....	20
DAFTAR PUSTAKA	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengelolaan data yang efisien dan terstruktur merupakan fondasi utama dalam pengembangan perangkat lunak modern. Seiring meningkatnya kompleksitas sistem, pengembang tidak hanya dituntut memahami cara menyimpan data, tetapi juga mampu memisahkan logika penggunaan dari detail implementasi. Konsep *Abstract Data Type* (ADT) hadir sebagai solusi melalui prinsip abstraksi yang mendefinisikan tipe data berdasarkan perilaku dan operasi yang diizinkan, bukan representasi memorinya. Dengan memisahkan antarmuka (*interface*) dari implementasi, ADT mendukung *information hiding*, enkapsulasi, serta membentuk sistem yang modular, *reusable*, dan mudah diuji.

Meskipun bersifat teoretis, pemahaman ADT memerlukan implementasi langsung untuk menguji kebenaran operasi, menangani kondisi batas (*boundary conditions*), dan memvalidasi kontrak spesifikasi yang ditetapkan. Penguasaan konsep ini juga menjadi prasyarat penting sebelum mempelajari algoritma lanjutan maupun arsitektur perangkat lunak berbasis komponen. Oleh karena itu, praktikum ini dilaksanakan untuk memberikan pengalaman langsung dalam merancang, mengimplementasikan, dan menguji ADT sesuai spesifikasi teknis.

1.2 Tujuan

Tujuan dari dilakukannya praktikum ini adalah

1. Memahami struktur ADT Stack
2. Memahami langkah-langkah menulis ADT Stack

1.3 Manfaat

Manfaat yang diperoleh dari praktikum ini adalah

1. Meningkatkan pemahaman tentang ADT Stack
2. Meningkatkan pemahaman bagaimana cara menulis ADT Stack

BAB II

PEMBAHASAN

2.1 Landasan teori

Struktur data adalah cara penyimpanan, pengorganisasian, dan pengaturan data di dalam media penyimpanan komputer sehingga data tersebut dapat digunakan secara efisien. Sedangkan data adalah representasi dari fakta dunia nyata. Fakta atau keterangan tentang kenyataan yang disimpan, direkam atau direpresentasikan dalam bentuk tulisan, suara, gambar, sinyal atau simbol.

Definisi ADT (Abstract Data Type) sendiri dalam struktur data merupakan sebuah spesifikasi atau kontrak yang mendefinisikan antarmuka suatu tipe data. ADT tidak menentukan bagaimana tipe data diimplementasikan, melainkan berfokus pada operasinya dan perilaku yang diharapkan dari tipe data tersebut.

ADT stack adalah model abstrak yang mengatur urutan elemen-elemen dalam urutan terbalik. Dalam stack, elemen yang ditambahkan terakhir akan dihapus terlebih dahulu. Ini berarti bahwa stack mendukung dua operasi utama: push yang menambahkan elemen ke bagian atas stack, dan pop yang menghapus elemen dari bagian atas stack. Stack memiliki aplikasi yang luas dalam berbagai konteks, termasuk fungsi stack dalam eksekusi program, pemulihan elemen, dan evaluasi/pengujian ekspresi

2.2 Langkah Pengerjaan

- NilaiMaksimum_2511532017
 1. Buka eclipse dan buat package baru dengan nama pekan3_2511532017 dan class NilaiMaksimum_2511532017.
 2. Setelah dibuat, kita akan membuat ADT nilai maksimum

```
package pekan3_2511532017;
import java.util.*;
public class NilaiMaksimum_2511532017 {
    public static int max_2017(Stack<Integer>s_2017) {
        Stack<Integer> backup = new Stack<Integer>();
        int MaxValue_2017 = s_2017.pop();
        backup.push (MaxValue_2017);
    }
}
```

3. Kita akan membuat method max, Stack<Integer> backup = new Stack<Integer>(); untuk membuat stack sementara untuk

menyimpan elemen yang dikeluarkan dari stack utama, agar stack utama tidak rusak.

```
package pekan3_2511532017;
import java.util.*;
public class NilaiMaksimum_2511532017 {
    public static int max_2017(Stack<Integer>s_2017) {
        Stack<Integer> backup = new Stack<Integer>();
        int MaxValue_2017 = s_2017.pop();
        backup.push (MaxValue_2017);
```

4. `int MaxValue_2017 = s_2017.pop();` mengeluarkan elemen teratas dan mejadikan sebagai kandidat nilai max. `backup.push(MaxValue_2017);` menyimpan elemen tersebut ke stack cadangan

```
        backup.push (MaxValue_2017);
        while (!s_2017.isEmpty()) {
            int next_2017 = s_2017.pop();
            backup.push(next_2017);
            MaxValue_2017 = Math.max(MaxValue_2017, next_2017);
        }while (!backup.isEmpty()) {
            s_2017.push(backup.pop());
        }
        return MaxValue_2017;
    }
}
```

5. selama stack utama masih berisi data, maka terus akan mengeluarkan elemen berikutnya, menyimpan ke cadangan, membandingkan dengan nilai max sekarang. Lalu memperbarui nilai max jika nilai lebih besar.

```
        backup.push (MaxValue_2017);
        while (!s_2017.isEmpty()) {
            int next_2017 = s_2017.pop();
            backup.push(next_2017);
            MaxValue_2017 = Math.max(MaxValue_2017, next_2017);
        }while (!backup.isEmpty()) {
            s_2017.push(backup.pop());
        }
        return MaxValue_2017;
    }
}
```

6. Memindahkan semua data dari backup ke stack ke s_2017 dengan urutan sama persis

```
        }
        return MaxValue_2017;
    }
}
```

7. Mengembalikan nilai max yang ditemukan

```
public static void main (String[]args) {
    Stack<Integer>s_2017 = new Stack<Integer>();
```

8. Membuat object stack yang hanya boleh menyimpan tipe data integer

```

        s_2017.push(70);
        s_2017.push(12);
        s_2017.push(20);
    }
}

```

9. Menambahkan elemen ke stack

```

        System.out.println("isi stack "+ s_2017);
        System.out.println("stack teratas "+ s_2017.peek());
        System.out.println("nilai maksimum "+ max_2017(s_2017));
    }
}

```

10. Memanggil method toString()

```

        System.out.println("isi stack "+ s_2017);
        System.out.println("stack teratas "+ s_2017.peek());
        System.out.println("nilai maksimum "+ max_2017(s_2017));
    }
}

```

11. Memunculkan elemen teratas tanpa menghapusnya

```

        System.out.println("isi stack "+ s_2017);
        System.out.println("stack teratas "+ s_2017.peek());
        System.out.println("nilai maksimum "+ max_2017(s_2017));
    }
}

```

12. Memanggil method pencarian dan nilai maks dicetak.

13. Output

```

isi stack [10, 70, 12, 20]
stack teratas 20
nilai maksimum 70
|

```

- SiswaStack_2511532017

1. Kita akan membuat class baru untuk SiswaStack_2511532017

```

class Siswa_2511532017 {
    String nama_2017;
    int nim_2017;

    public Siswa_2511532017 (String nama_2017, int nim_2017) {
        this.nama_2017 = nama_2017;
        this.nim_2017 = nim_2017;
    }

    public String toString() {
        return "Nim : " + nim_2017 + ", Nama : " + nama_2017;
    }
}

```

2. Digunakan untuk menyimpan objek di stack, nama_2511532017 dan nim_2511532017 untuk menyimpan data mahasiswa. toString merupakan method override, method ini akan otomatis terpanggil saat objek dicetak.


```
public class SiswaStack_2511532017 {
    private ArrayList<Siswa_2511532017>stack;

    public SiswaStack_2511532017 () {
        stack = new ArrayList<>();
    }
}
```

3. Kelas tempat mengimplementasikan stack, private ArrayList<Siswa_2511532017> stack; wadah untuk menyimpan elemen.

```
public SiswaStack_2511532017 () {
    stack = new ArrayList<>();
}
```

4. Ini constructor kelas SiswaStack_2511532017 yang akan otomatis terpanggil saat membuat objek baru dengan kata kunci new

```
public void push (Siswa_2511532017 mhs) {
    stack.add(mhs);
}
public Siswa_2511532017 pop() {
    if(!isEmpty()) {
        return stack.remove(stack.size()-1);
    }return null;
}
```

5. Menambahkan objek ke akhir ArrayList. Pada stack, elemen yang baru ditambahkan selalu menjadi top.

```
public void push (Siswa_2511532017 mhs) {
    stack.add(mhs);
}
public Siswa_2511532017 pop() {
    if(!isEmpty()) {
        return stack.remove(stack.size()-1);
    }return null;
}
```

6. Jika stack tidak kosong, maka akan menghapus elemen terakhir (size()-1), jika stack kosong akan mengembalikan nilai null

```

public Siswa_2511532017 peek() {
    if(!isEmpty()) {
        return stack.get(stack.size()-1);
    }return null;
}
public boolean isEmpty() {
    return stack.isEmpty();
}

```

7. Menampilkan elemen tanpa menghapusnya

```

public Siswa_2511532017 peek() {
    if(!isEmpty()) {
        return stack.get(stack.size()-1);
    }return null;
}
public boolean isEmpty() {
    return stack.isEmpty();
}

```

8. Pengecekan apakah stack masih berisi data

```

public void tampilkanSiswa_2511532017() {
    for (int i = stack.size()-1; i>=0; i--) {
        System.out.println(stack.get(i));
    }
}

```

9. Loop mundur dari index terakhir ke 0, agar sesuai dengan prinsip LIFO

```

public static void main(String[]args) {
    SiswaStack_2511532017 studentStack = new SiswaStack_2511532017();

    Siswa_2511532017 mhs1 = new Siswa_2511532017 ("Ali", 1);
    Siswa_2511532017 mhs2 = new Siswa_2511532017 ("Boby", 2);
    Siswa_2511532017 mhs3 = new Siswa_2511532017 ("Charles", 3);
}

```

10. Menyimpan objek di memori yang berbeda

```

studentStack.push(mhs1);
studentStack.push(mhs2);
studentStack.push(mhs3);

```

11. Memasukkan mhs1,mhs2,mhs3 ke stack

```

System.out.println("siswa dalam stack ");
studentStack.tampilkanSiswa_2511532017();

```

12. Menampilkan dari index 2 ke 0

```

        System.out.println("siswa teratas : "+ studentStack.peek() );
        System.out.println("mengeluarkan siswa teratas dari stack : "+ studentStack.pop());
        studentStack.tampilkanSiswa_2511532017();
    }
}

```

13. Mengembalikan referensi objek Charles (stack tetap utuh).

```

        System.out.println("siswa teratas : "+ studentStack.peek() );
        System.out.println("mengeluarkan siswa teratas dari stack : "+ studentStack.pop());
        studentStack.tampilkanSiswa_2511532017();
    }
}

```

14. Menghapus & mengembalikan Charles. Stack menjadi: [Ali, Bobby]

```

        System.out.println("siswa teratas : "+ studentStack.peek() );
        System.out.println("mengeluarkan siswa teratas dari stack : "+ studentStack.pop());
        studentStack.tampilkanSiswa_2511532017();
    }
}

```

15. Cetak ulang

16. Output

```

siswa dalam stack
Nim : 3, Nama : Charles
Nim : 2, Nama : Bobby
Nim : 1, Nama : Ali
siswa teratas : Nim : 3, Nama : Charles
mengeluarkan siswa teratas dari stack : Nim : 3, Nama : Charles
Nim : 2, Nama : Bobby
Nim : 1, Nama : Ali

```

- stackArray_2511532017
 1. kita akan membuat class baru untuk stackArray_2511532017
 2. Buat class baru dengan nama stackArray_2511532017

```

package pekan3_2511532017;

public class stackArray_2511532017 {
    static final int max_2017 = 1000;
    int top_2017;
    int a[] = new int [max_2017];
    boolean isEmpty()
    {
        return (top_2017 < -1);
    }
    stackArray_2511532017(){
        top_2017 = -1;
    }
}

```

3. Kapasitas max stack 1000, lalu pointer untuk menunjuk elemen teratas stack

```

package pekan3_2511532017;

public class stackArray_2511532017 {
    static final int max_2017 = 1000;
    int top_2017;
    int a[] = new int [max_2017];
    boolean isEmpty()
    {
        return (top_2017 < -1);
    }
    stackArray_2511532017() {
        top_2017 = -1;
    }
}

```

4. Array primitif sebagai tempat menyimpan dengan ukuran tetap

```

package pekan3_2511532017;

public class stackArray_2511532017 {
    static final int max_2017 = 1000;
    int top_2017;
    int a[] = new int [max_2017];
    boolean isEmpty()
    {
        return (top_2017 < -1);
    }
    stackArray_2511532017() {
        top_2017 = -1;
    }
}

```

5. Constructor yang berjalan saat new stackArray_2511532017() dipanggil

```

package pekan3_2511532017;

public class stackArray_2511532017 {
    static final int max_2017 = 1000;
    int top_2017;
    int a[] = new int [max_2017];
    boolean isEmpty()
    {
        return (top_2017 < -1);
    }
    stackArray_2511532017() {
        top_2017 = -1;
    }
}

```

6. Method mengecek apakah stack kosong

```

boolean push(int x) {
    if(top_2017 >= (max_2017 - 1)) {
        System.out.println("stack overflow");
        return false;
    } else {
        a[++top_2017] = x;
        System.out.println(x + " dimasukkan ke dalam stack");
        return true;
    }
}

```

7. Menambahkan elemen, jika stack sudah 999 maka array penuh dan gagal menambahkan, return false

```

boolean push(int x) {
    if(top_2017 >= (max_2017 - 1)) {
        System.out.println("stack overflow");
        return false;
    } else {
        a[++top_2017] = x;
        System.out.println(x + " dimasukkan ke dalam stack");
        return true;
    }
}

```

8. Proses menambahkan dengan index naik terus, retrun true jika berhasil

```

int pop() {
    if(top_2017 < 0) {
        System.out.println("stack underflow");
        return 0;
    } else {
        int x = a[top_2017--];
        return x;
    }
}

```

9. Mengambil nilai teratas, jika stack kosong cetak pesan, jika ada ambil nilai teratas dan return x

```

int peek() {
    if (top_2017 < 0) {
        System.out.println("stack underflow");
        return 0;
    } else {
        int x = a[top_2017];
        return x;
    }
}

```

10. Menampilkan elemen tanpa menghapus

```

void print () {
    for (int i = top_2017; i>-1;i--) {
        System.out.print(" "+ a[i]);
    }
}

```

11. Loop mundur dari top ke 0

- stackArrayDriver_2511532017
 1. Kita akan membuat class driver untuk stack array
 2. Buat class dengan nama stackArrayDriver_2511532017

```
package pekan3_2511532017;

public class stackArrayDriver_2511532017 {

    public static void main(String[] args) {
        stackArray_2511532017 s = new stackArray_2511532017();
```

3. Memanggil cosntructor dari kelas sebelumnya

```
-----
s.push(10);
s.push(20);
s.push(30);
```

4. Memasukkan elemen pada stack

```
System.out.println(s.pop()+" dikeluarkan dari stack");
System.out.println("elemen teratas adalah"+ s.peek());
System.out.print("elemen pada stack : ");
s.print();
}
```

5. Mengeluarkan elemen atas

```
System.out.println(s.pop()+" dikeluarkan dari stack");
System.out.println("elemen teratas adalah"+ s.peek());
System.out.print("elemen pada stack : ");
s.print();
}
```

6. Menampilkan elemen teratas saat ini

```
System.out.println(s.pop()+" dikeluarkan dari stack");
System.out.println("elemen teratas adalah"+ s.peek());
System.out.print("elemen pada stack : ");
s.print();
}
```

7. Menampilkan semua elemen

8. Output

```
10 dimasukkan ke dalam stack
20 dimasukkan ke dalam stack
30 dimasukkan ke dalam stack
30 dikeluarkan dari stack
elemen teratas adalah20
elemen pada stack : 20 10
```

- stackPostfix_2511532017

1. Kita akan membuat class baru untuk stacPostfix_2511532017

```
package pekan3_2511532017;

import java.util.*;

public class stackPostfix_2511532017 {
    public static int postfixEvaluate_2511532017(String expression) {
        Stack<Integer> s = new Stack<Integer>();
        Scanner input = new Scanner (expression);
```

2. Method ini bertugas menghitung hasil ekspresi matematika dalam notasi Postfix (angka muncul sebelum operator)

```
package pekan3_2511532017;

import java.util.*;

public class stackPostfix_2511532017 {
    public static int postfixEvaluate_2511532017(String expression) {
        Stack<Integer> s = new Stack<Integer>();
        Scanner input = new Scanner (expression);
```

3. Stack<Integer> s bertugas unuk menyimpan operand (angka)

```
package pekan3_2511532017;

import java.util.*;

public class stackPostfix_2511532017 {
    public static int postfixEvaluate_2511532017(String expression) {
        Stack<Integer> s = new Stack<Integer>();
        Scanner input = new Scanner (expression);
```

4. Scanner input = new Scanner(expression); memecah string expression menjadi token (kata/angka terpisah spasi)

```
        while (input.hasNext()) {
            if(input.hasNextInt()) {
                s.push(input.nextInt());
            }else {
                String operator_2017 = input.next();
                int operand2 = s.pop();
                int operand1 = s.pop();
                if(operator_2017.equals("+")) {
                    s.push(operand1 + operand2);
                }else if (operator_2017.equals("-")) {
                    s.push (operand1 - operand2);
                }else if (operator_2017.equals("*")) {
                    s.push (operand1 * operand2);
                }else {
                    s.push(operand1 / operand2);
                }
            }
        }
```

5. Loop berjalan selama masih ada token yang belum dibaca. input.hasNextInt() mengecek apakah token berikutnya adalah bilangan bulat, jika true s.push(input.nextInt()) angka dibaca langsung dan dimasukkan ke stack, jika false berarti token tersebut adalah operator

```
String operator_2017 = input.next();
int operand2 = s.pop();
int operand1 = s.pop();
```

6. operand2 = s.pop() mengambil elemen teratas (angka yang baru dimasukkan). operand1 = s.pop() mengambil elemen dibawahnya (angka yang dimasukkan lebih dahulu)

```
}else {
String operator_2017 = input.next();
int operand2 = s.pop();
int operand1 = s.pop();
if(operator_2017.equals("+")) {
s.push(operand1 + operand2);
}else if (operator_2017.equals("-")) {
s.push (operand1 - operand2);
}else if (operator_2017.equals("*")) {
s.push (operand1 * operand2);
}else {
s.push(operand1 / operand2);
}
```

7. mengecek string operator menggunakan .equals() karena tidak boleh menggunakan (==) untuk membandingkan isi String

```
}input.close();
return s.pop();
}
public static void main (String[]args) {
System.out.println("hasil positifx = "+ postfixEvaluate_2511532017("5 2 4 * + 7 -"));
}
}
```

8. memanggil method dengan ekspresi "5 2 4 * + 7 -"
 9. output
-
- hasil positifx = 6

- Website_2511532017

1. Kita buat kelas untuk Webstie_2511532017

```
package pekan3_2511532017;

public class Website_2511532017 {
private String judul_2017;
private String URL_2017;
```

2. Private digunakan agar tidak bisa diakses langsung dari luar kelas


```
//constructor
public Website_2511532017(String judul_2017, String URL_2017){
    this.judul_2017 = judul_2017;
    if (URL_2017 == null || URL_2017.trim().isEmpty()) {
        System.out.println("X Error: URL tidak boleh kosong!");
    } else {
        this.URL_2017 = URL_2017.trim();
    }
}
```

3. Menginisialisasi objek dengan parameter judul_2017 dan URL_2017, untuk url saya mengecek apakah url itu null, jika kosong cetak pesan error

```
//getter dan setter
public String getJudul_2017() {return judul_2017;}
public void setJudul_2017(String judul_2017) {this.judul_2017 = judul_2017;}
```

4. Mengembalikan nilai judul website dan mengatur nilai judul (tanpa validasi)

```
public String getURL_2017() {return URL_2017;}
public void setURL_2017(String URL_2017) {
    if (URL_2017== null || URL_2017.trim().isEmpty()) {
        System.out.println("URL tidak boleh kosong");
    } else {this.URL_2017 = URL_2017.trim();}
}
```

5. Mengembalikan nilai judul website dan mengatur nilai judul dengan validasi, jika url tidak valid akan mencetak pesan error

```
@Override
public String toString() {
    return "\n Nama Website: " + judul_2017 +
        "\nURL : "+ URL_2017;
}
```

6. Mengembalikan representasi teks dari objek untuk memudahkan pencetakan/debugging dengan Menimpa method toString() bawaan dari kelas Object

- Browser_2511532017

1. Kita akan membuat driver untuk Website dengan nama class Browser_2511532017

```

package pekan3_2511532017;

import java.util.ArrayList;
import java.util.Scanner;

public class Browser_2511532017 {
    private ArrayList<Website_2511532017>stack;
    public Browser_2511532017 () {
        stack = new ArrayList<>();
    }
}

```

2. class Browser_2511532017 bertindak sebagai controller yang mengelola navigasi browser

```

    private ArrayList<Website_2511532017>stack;
    public Browser_2511532017 () {
        stack = new ArrayList<>();
    }
}

```

3. private ArrayList<Website_2511532017> stack; digunakan untuk menyimpan tumpukan objek Menginisialisasi objek Browser dengan membuat ArrayList kosong.

```

//method menu
public static void tampilkanMenu_2511532017 () {
    System.out.println("\n== Browser History NIM: 2511532017 ==");
    System.out.println("1. Kunjungi Website (Push)");
    System.out.println("2. Tombol Back (Pop)");
    System.out.println("3. Lihat Halaman Aktif (Peek)");
    System.out.println("4. Cek Status History");
    System.out.println("5. keluar");
}
}

```

4. Method untuk menampilkan pilihan menu operasi browser ke layar.

```

//memasukkan website baru
public void tambahWebsite_2511532017(Scanner sc) {
    System.out.print("Masukkan Judul Website : ");
    String judul = sc.nextLine();

    String url;
    while (true) {
        System.out.print("Masukkan URL: ");
        url = sc.nextLine();
        if (!url.trim().isEmpty()) {
            break;
        }
        System.out.println("Error: URL tidak boleh kosong! Silakan ulangi.");
    }
    Website_2511532017 webBaru = new Website_2511532017(judul, url);
    this.push(webBaru); //push ke stack
    System.out.println(" Website berhasil ditambahkan ke history");
}

public void push (Website_2511532017 wbst) {
    stack.add(wbst);
}
}

```

5. Method untuk menambahkan website yang akan dikunjungi, Menggunakan while(true) untuk memaksa pengguna memasukkan URL yang tidak kosong dan jika valid loop berhenti

```

        Website_2511532017 webBaru = new Website_2511532017(judul, url);
        this.push(webBaru);    //push ke stack
        System.out.println(" Website berhasil ditambahkan ke history");
    }
    public void push (Website_2511532017 wbst) {
        stack.add(wbst);
    }

```

6. Menambahkan objek website ke akhir ArrayList (dianggap sebagai "atas" stack)

```

//mengeluarkan website
public Website_2511532017 pop() {
    if (!stack.isEmpty()) {    //cek stack kosong atau tidak
        return stack.remove(stack.size() - 1);}
    return null;}

```

7. Method untuk menghapus, dicek dahulu apakah stack tidak kosong, jika data tidak kosong maka akan dihapus dan kembalian ke elemen di indeks terakhir (size()-1)

```

//melihat website tanpa menghapus
public Website_2511532017 peek() {
    if(!isEmpty()) {
        return stack.get(stack.size()-1);}
    return null;}

public boolean isEmpty() {
    return stack.isEmpty();}

```

8. Method untuk melihat website tanpa menghapus

```

public boolean isEmpty() {
    return stack.isEmpty();}

```

9. meneruskan pemanggilan ke method isEmpty() milik ArrayList.

```

//method cek status history
public void cekStatusHistory_2511532017() {
    int total = stack.size();
    System.out.println("\nJumlah riwayat history : " + total);

    if (total == 0) {
        System.out.println("History masih kosong");
    } else {
        System.out.println("== Daftar Riwayat ==");
        for (int i = total - 1; i >= 0; i--) {
            System.out.println((i == total - 1 ? " Aktif " : " ") + stack.get(i));
        }
    }
}

```

10. Method cek status history, jika stack masih kosong maka akan menampilkan pesan kosong, melakukan iterasi dari belakang ke 0 dan memberi label Aktif untuk website yang sedang dibuka (puncak)

```

public static void main(String[] args) {
    Browser_2511532017 browser = new Browser_2511532017();
    Scanner sc = new Scanner(System.in);
    int pilihan;

    do {
        tampilkanMenu_2511532017();
        System.out.print("Pilih menu (1-5): ");
        try {
            pilihan = Integer.parseInt(sc.nextLine().trim());
        } catch (NumberFormatException e) {
            pilihan = -1;
            System.out.println("Input harus angka 1-5!");
            continue;
        }
    }
}

```

11. Do while digunakan untuk memastikan menu ditampilkan minimal sekali dan terus berulang sampai pengguna memilih opsi 5 (keluar).

```

        System.out.print("Pilih menu (1-5): ");
        try {
            pilihan = Integer.parseInt(sc.nextLine().trim());
        } catch (NumberFormatException e) {
            pilihan = -1;
            System.out.println("Input harus angka 1-5!");
            continue;
        }
    }
}

```

12. Menggunakan try-catch untuk menangani input non-angka, mencegah program crash (InputMismatchException).

```

        switch (pilihan) {
            case 1:
                browser.tambahWebsite_2511532017(sc);
                break;
            case 2:
                Website_2511532017 webBack = browser.pop();
                if (webBack != null) {
                    System.out.println("Berhasil dihapus: " + webBack.getJudul_2017());
                } else {
                    System.out.println("History kosong, Tidak bisa Back.");
                }
                break;
            case 3:
                Website_2511532017 webActive = browser.peek();
                if (webActive != null) {
                    System.out.println("Halaman Aktif:\n" + webActive);
                } else {
                    System.out.println("Tidak ada halaman yang dibuka.");
                }
                break;
            case 4:
                browser.cekStatusHistory_2511532017();
                break;
            case 5:
                break;
            default:
                System.out.println(" Pilihan tidak valid.");
        }
    } while (pilihan != 5);

    sc.close();
}
}

```

13. Switch-Case digunakan untuk menentukan pilihan user

14. Output input website baru

```
=== Browser History NIM: 2511532017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. keluar
Pilih menu (1-5): 1
Masukkan Judul Website : youtube
Masukkan URL: youtube.com
Website berhasil ditambahkan ke history

=== Browser History NIM: 2511532017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. keluar
Pilih menu (1-5): 1
Masukkan Judul Website : instagram
Masukkan URL: instagram.com
Website berhasil ditambahkan ke history

=== Browser History NIM: 2511532017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. keluar
Pilih menu (1-5):
```

15. Output lihat halaman aktif

```
=== Browser History NIM: 2511532017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. keluar
Pilih menu (1-5): 3
Halaman Aktif:

Nama Website: instagram
URL : instagram.com
```

16. Cek status history

```

=== Browser History NIM: 2511532017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. keluar
Pilih menu (1-5): 4

Jumlah riwayat history : 2
=== Daftar Riwayat ===
Aktif
Nama Website: instagram
URL : instagram.com

Nama Website: youtube
URL : youtube.com

```

17. Tombol back

```

=== Browser History NIM: 2511532017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. keluar
Pilih menu (1-5): 2
Berhasil dihapus: instagram

```

18. Output Url Error

```

=== Browser History NIM: 2511532017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. keluar
Pilih menu (1-5): 1
Masukkan Judul Website : facebook
Masukkan URL:
Error: URL tidak boleh kosong! Silakan ulangi.
Masukkan URL:

```

BAB III

KESIMPULAN

Struktur data Stack berhasil diimplementasikan menggunakan ArrayList dengan prinsip LIFO (Last In First Out). Tiga operasi fundamental stack yaitu push (menambahkan website), pop (tombol back), dan peek (melihat halaman aktif) berfungsi sesuai ekspektasi. Enkapsulasi diterapkan dengan baik melalui penggunaan akses modifier private pada atribut dan penyediaan method *getter/setter* untuk akses terkontrol.

DAFTAR PUSTAKA

- [1] A. Chauhan, "Abstract Data Types," GeeksforGeeks, Jan. 31, 2026.
[Online]. Available: <https://www.geeksforgeeks.org/dsa/abstract-data-types/>. [Accessed: Apr. 18, 2026].